RookGuard — Environment inventory, CVE scanning, and guided remediation as a Hermes Agent plugin

Nov 5, 2026 · @Hane

Prepared for the Hermes Agent Plugin Hackathon (submission deadline: Nov 11, 2026, 5 PM EST). Author: Maxime Vaillard (309nahe). Scaffold repository: https://github.com/309nahe/hermes-hackathon-plugin

1. Summary

RookGuard is a general-purpose Hermes plugin that keeps an inventory of the packages installed on the host where the agent runs, scans that inventory nightly against cve.now, and presents prioritized findings with proposed fixes at the next session. The user approves upgrades per package or in bulk, and RookGuard runs them. A request like "install code-server and expose it" becomes a tracked, auditable, self-healing action.

Tagline: track, scan, fix.

2. Hackathon context

- 7 days, $20,000 in cash prizes. Plugins extend Hermes with tools, hooks, slash commands, CLI commands, bundled skills and specialized kinds.
- Judges score usefulness, reliability, polish and creativity, and favor plugins that do more than a prompt or a skill could.
- Submission requires three things:
  1. A PR to NousResearch/hermes-agent adding `plugin-catalog/<name>.yaml`, pinning a full 40-character commit SHA.
  2. A demo video under 90 seconds on X, tagging @NousResearch.
  3. The Typeform submission, before Nov 11, 5 PM EST.

3. Author and motivation

Maxime Vaillard has a 5-year SOC background at DGFIP, where he manages 10 analysts. RookGuard reflects his "do good" values: practical security hygiene for homelabs and small SOCs, where agents increasingly install software that nobody tracks afterwards.

4. Why RookGuard can win

- **Beyond a prompt or skill:** it combines hooks (`post_tool_call` , `on_session_start`), LLM-callable tools, durable state, slash commands and a bundled skill.
- **Reliability:** cve.now is a real queryable catalog. Its `check_stack` accepts a package list and returns affected CVEs with fixed versions, so no data is fabricated.
- **Polish:** a clean slash-command UX, severity grouping, and a morning briefing injected into context.
- **Creativity:** a security-hygiene copilot for homelabs and SOCs.
- **Demo-ability:** pre-seed a vulnerable package, scan, show findings, fix one, all in 90 seconds.

5. Architecture

Plugin kind: general (not a specialized kind). It registers tools, hooks, slash commands and a bundled skill.

Surface	Component	Role
Storage	`plugin_db('rookguard')`	SQLite for inventory, scans and findings (durable)
Storage	`plugin_data_dir('rookguard')`	Exported reports
Hook	`post_tool_call`	Detects install/upgrade commands (`apt-get install` , `pip install` , `npm install -g` , `cargo install` , …) and appends package, version and source to the inventory
Hook	`on_session_start`	If the last scan is older than 12 h or findings are unaddressed, injects a short briefing into context
Tool	`rookguard_inventory`	Snapshot the environment
Tool	`rookguard_scan`	Build the package list, call cve.now, store findings
Tool	`rookguard_status`	Findings grouped by severity, with fixed versions

Surface	Component	Role
Tool	`rookguard_fix`	Run the upgrade for one package or all approved ones
Slash command	`/rookguard inventory|scan|status|fix [pkg|all]`	User-facing UX
Skill	`skills/vuln-triage/SKILL.md`	How to read findings, prioritize by KEV, CVSS and active exploitation, and decide between fix and accept-risk

External dependency: the cve.now MCP server (`mcp__cve_now__check_stack`). The plugin probes its availability through `ctx.dispatch_tool` and degrades gracefully: it prints install instructions and falls back to direct HTTPS calls to the cve.now API.

6. MVP scope

In scope (week 1)

- Inventory snapshot (dpkg, pip, npm) plus incremental hook tracking.
- Scan via cve.now `check_stack`.
- Status report grouped by severity with fixed versions.
- Fix command per package and in bulk, with user approval (sudo allowed).
- Morning briefing via `on_session_start` context injection.
- Bundled `vuln-triage` skill.
- README disclosure section.

Out of scope (TODOs)

- Gateway push notification.
- Switch between sudo and user-level fixes (sudo for now, configurable later).
- OS coverage beyond Debian/Ubuntu, pip and npm.
- Auto-approval of upgrades (never in v1).
- Web dashboard and email digest.

Decisions taken

1. Fix execution uses sudo by default, always user-approved. A user-level-only switch is a TODO.
2. Inventory combines a snapshot baseline with hook-tracked deltas.
3. The briefing is delivered by context injection first. Gateway push is a TODO.

7. Implementation plan (6 days)

Day 1 — Storage and inventory

1. Create the skeleton under `rookguard/`: `plugin.yaml`, `__init__.py`, `schemas.py`, `tools.py`, `skills/`.
2. Build the SQLite store via `plugin_db('rookguard')` with tables `packages`, `scans`, `findings`.
3. Implement the `rookguard_inventory` snapshot (`dpkg -l`, `pip list --format=json`, `npm ls -g --json`) and `/rookguard inventory`.
4. Add the `post_tool_call` hook to detect install and upgrade commands and record deltas.

Day 2 — CVE scan

5. Implement `rookguard_scan`: build the package list, call `ctx.dispatch_tool('mcp__cve_now__check_stack', ...)`, parse `affected[]` with fixed versions, store findings.
6. Handle a missing cve.now MCP gracefully (probe, then fallback message).
7. Implement `rookguard_status` with severity grouping.

Day 3 — Remediation

8. Implement `rookguard_fix [pkg|all]`: build the upgrade command per ecosystem, run it via the terminal with approval, record the outcome.
9. Use sudo by default, with a TODO for the user-level switch.

Day 4 — Briefing and skill

10. Implement the `on_session_start` briefing (staleness check and context injection).
11. Write the bundled `vuln-triage` skill.

Day 5 — Polish and tests

12. Unit tests for parsers and handlers, loading modules via `importlib` to avoid `tools` shadowing.
13. Integration test: snapshot plus scan against a seeded vulnerable package.
14. README disclosure section and Makefile (CI is already in place).

Day 6 — Demo and submission

15. Pre-seed a vulnerable package (for example `requests==2.28.0` in a venv).
16. Record a demo of 90 seconds or less: inventory, scan, findings, fix one.
17. Commit, push, obtain the SHA, and update `plugin-catalog-entry.yaml`.

18. Open the PR to NousResearch/hermes-agent adding `plugin-catalog/rookguard.yaml`.
19. Post the demo on X tagging @NousResearch, then submit the Typeform.

8. Demo plan (90 seconds or less)

#	Step	Time
1	`/rookguard inventory` captures the box	5 s
2	Pre-seeded vulnerable package `requests` 2.28.0	5 s
3	`/rookguard scan`: cve.now returns findings with CVE IDs and fixed versions	25 s
4	`/rookguard status`: findings grouped by severity	15 s
5	`/rookguard fix requests`: upgrade runs with approval	20 s
6	Morning briefing at the next session start	10 s
7	Closing line: "RookGuard - track, scan, fix."	10 s

9. Catalog submission checklist

- [ ] Public repo with plugin, README and `plugin.yaml`.
- [ ] `hermes plugins validate . --install-deps` passes (CI is wired).
- [ ] Full 40-character SHA pinned in `plugin-catalog/rookguard.yaml`.
- [ ] PR to NousResearch/hermes-agent (one file added).
- [ ] Demo video under 90 seconds on X tagging @NousResearch.
- [ ] Both links submitted via the Typeform before Nov 11, 5 PM EST.

10. Disclosure (catalog rule 13)

Item	Disclosure
Network calls	cve.now API (via MCP tool or direct HTTPS), during scans only

Item	Disclosure
Shell commands	`dpkg`, `pip`, `npm` for inventory; `apt-get`, `pip`, `npm` upgrades for fixes (user-approved, sudo allowed)
Background processes	None in v1
Stored credentials	None (cve.now is unauthenticated)
Telemetry	None; opt-in only if ever added

11. Risks and mitigations

Risk	Mitigation
Inventory parsing is fragile across OS versions	Scope v1 to Debian/Ubuntu, pip and npm
cve.now MCP not installed	Probe, degrade gracefully with clear instructions, fall back to direct HTTPS
Fix commands on system packages can break the host	Propose first, require explicit approval, default to per-package fixes
Demo timing	Pre-seed data; avoid live, network-heavy upgrades

12. Technical notes for implementation

- The plugin loads as `hermes_plugins.<slug>`. Intra-plugin imports must be relative (`from .schemas import ...`); an absolute `from tools import` resolves to Hermes's own `tools` package.
- Quote the manifest `version` in YAML, otherwise `0.1.0` parses as a number.
- `requires_hermes` is a SemVer floor (`>=0.22.0`), never CalVer and never newer than the current release.
- `provides_tools` and `provides_hooks` must exactly match what `register()` registers.
- The security scan flags the literal "curl | bash" even inside echo strings, so rephrase it.
- Run tests from a neutral working directory (not the repo directory) so `tools.py` does not shadow Hermes's `tools` package. Load handler modules via `importlib.util.spec_from_file_location`.

- Bootstrap uses `hermes plugins enable` rather than manual YAML edits, which avoids a pyyaml dependency.
- Declared capabilities must match registrations; undeclared capability creep is treated as a security issue.
- Read secrets via `agent.secret_scope.get_secret`, never from `.env`.
- Store durable data with `plugin_data_dir` or `plugin_db`, never inside the install tree.